NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

MARCOS VINICIUS BONFIM ROMÃO

BSc in Electrical and Computer Engineering

# 5G NETWORKS: DEVELOPMENT OF A 5G CORE AND RADIO ACCESS NETWORK

## PRE-DISSERTATION INFORMATION

MASTER IN ELECTRICAL AND COMPUTER ENGINEERING

NOVA University Lisbon

***Draft:*** *May 20, 2026*

NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

# 5G NETWORKS: DEVELOPMENT OF A 5G CORE AND RADIO ACCESS NETWORK

## PRE-DISSERTATION INFORMATION

**MARCOS VINICIUS BONFIM ROMÃO**

BSc in Electrical and Computer Engineering

**Supervisor:** Rodolfo Oliveira
*Full Professor, NOVA University Lisbon*

MASTER IN ELECTRICAL AND COMPUTER ENGINEERING

NOVA University Lisbon

***Draft:*** *May 20, 2026*

# Contents

# List of Figures

# List of Tables

# Listings

# List of Listings

# Acronyms

**COTS** Commercial Off-The-Shelf *(p. 3)*

**eNodeB** Evolved Node B *(pp. 3, 14, 15)*
**EPC** Evolved Packet Core *(pp. 2, 3, 5, 7, 14)*

**LNA** Low Noise Amplifier *(p. 4)*

**RAN** Radio Access Network *(pp. 2–5, 12–15)*
**RSP** Remote SIM Provisioning *(p. 16)*

**SCTP** Stream Control Transmission Protocol *(p. 4)*
**SDR** Software-Defined Radio *(pp. 3, 4, 11)*

**UE** User Equipment *(pp. 3, 14, 15)*

# Bibliography

[1] *MongoDB Community Edition: Installation Guide for Linux*. URL: https://www.mongodb.com/docs/manual/administration/install-community/?operating-system=linux&linux-distribution=ubuntu&linux-package=default&search-linux=with-search-linux (cit. on p. 8).

[2] *Open5GS Guide: Quick Start*. URL: https://open5gs.org/open5gs/docs/guide/01-quickstart/ (cit. on p. 7).

# 4G Network Setup

Before developing the 5G network testbed, a 4G network was first established to serve as a performance baseline and a proof-of-concept for all hardware and software components. The 4G network setup was essential for several reasons:

- Validating that the chosen hardware and software stack worked correclty
- Establishing a known and well-documented reference point for network behavior
- Identifying potential deployment hurdles before scaling to 5G complexity

This appendix documents the complete 4G testbed architecture, deployment procedures, and validation results.

The network stack consists of two dedicated machines running the backbone infrastructure: one running Open5GS for the Evolved Packet Core (EPC), while the other runs srsRAN 4G for the Radio Access Network (RAN). Readers can use this section as a guide for replicating the network, and as a foundation for understanding the experimental setup that enabled the 5G network development described in the main thesis.

This section will provide a comprehensive guide on how the 4G network was established, including a description of the equipment used for the setup and the software chosen for the network stack, the necessary setup steps and specific configurations applied to optimize performance, and the testing methodologies employed to validate the network's functionality and performance.

As an effort to make the network as reproducible as possible, the setup process will be detailed step-by-step, starting from the initial hardware assembly to the final testing phase. This will include instructions on how to connect the various components, configure the network settings, and troubleshoot common issues that may arise during the setup process.

## .1 System Architecture Overview

The 4G testbed is a self-contained LTE system comprising three main functional domains:

- **RAN**: Handles air-interface communication using srsRAN Evolved Node B (eNodeB) software running on a dedicated machine, along with a Nuand BladeRF Software-Defined Radio (SDR) and appropriate antennas for signal transmission and reception
- **EPC**: Implements the core network functions using Open5GS software running on a separate machine, providing essential services such as authentication, user management, and data routing to the internet
- **User Equipment (UE)**: Physical Commercial Off-The-Shelf (COTS) device configured to connect to the RAN and access the services provided by the EPC

Figure .1 presents the logical architecture.

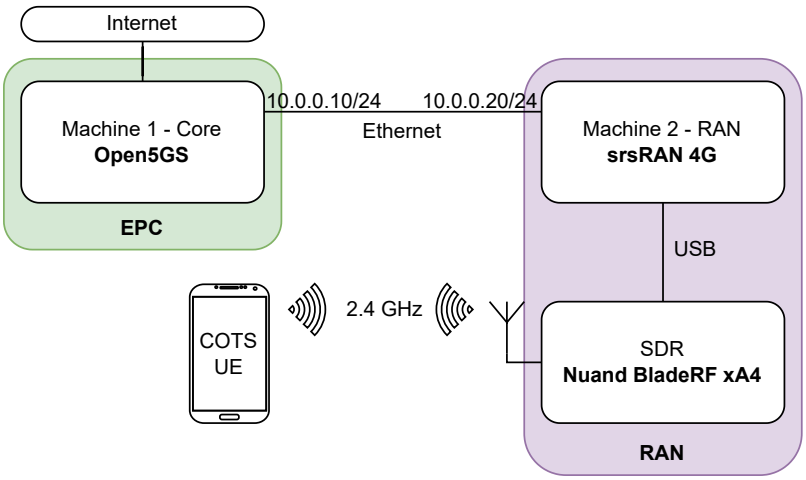


Figure .1: Logical architecture of the 4G network, illustrating the interactions between the RAN, EPC, and UE components.

## .2 Hardware Components

The hardware used for the 4G network setup is composed of two identical machines, a Software-Defined Radio and antennas.

The computers running the 4G network are DELL Optiplex Micro 7020, with the following specifications:

| Component | Machine 1 | Machine 2 |
|---|---|---|
| CPU | Intel Core i7-14700T | Intel Core i7-14700T |
| RAM | 16 GB DDR5 | 16 GB DDR5 |
| Operating System | Ubuntu 22.04 LTS | Ubuntu 22.04 LTS |
| Storage | 512 GB SSD | 512 GB SSD |

Table .2: Specifications of the machines used for the 4G network setup.

Data flows between the machines via a dedicated Ethernet backhaul network, composed of the two computers. The RAN on Machine 2 establishes a connection to the EPC on Machine 1 using Stream Control Transmission Protocol (SCTP).

The radio interface of the 4G network is implemented using a Nuand BladeRF xA4 SDR. This device connects the Machine 2 to provide the necessary radio capabilities for the RAN. In practical terms, the host machine executes the signal processing and network stack components, whereas the BladeRF xA4 manages the actual radio I/O. This separation of responsibilities improves flexibility, simplifies experimentation, and makes the overall testbed easier to implement andadapt to different deployment scenarios.

Attached to the xA4's SMA connectors are two Low Noise Amplifiers (LNAs), along with two omnidirectional antennas. These LNAs are used to amplify the TX/RX signals, improving the sensitivity of the RAN, expanding its coverage area and allowing for better performance in terms of signal quality.

| **Brand** | **Type** | **Model** | **Transmission** |
|---|---|---|---|
| Nuand | LNA | BT-100 | RX |
| Nuand | LNA | BT-200 | TX |

Table .3: Hardware components used for the radio interface in the 4G network setup.

Below, in Figure .2, shows the physical setup of the 4G network, including the two machines, the SDR, and the antennas. The photograph illustrates how the components are arranged in the lab environment, providing a visual reference for replicating the setup.

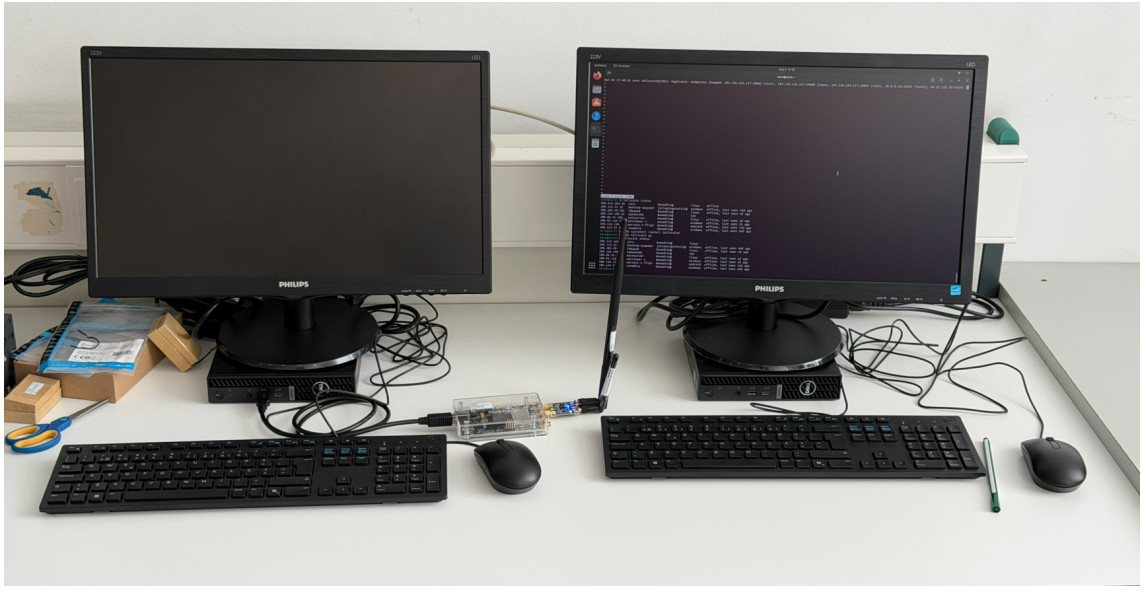


Figure .2: Photograph of the 4G network setup, showing the two machines, connected by an ethernet cable, the Nuand BladeRF xA4 SDR, and the antennas used for the radio interface.

## .3 Software Components

The software stack of the network is composed mainly of three components:

- Open5GS, for the core network functions, running on Machine 1.
- srsRAN 4G, for the RAN, running on Machine 2.
- SoapySDR, bridging the connection between the BladeRF board and the srsRAN software, running on Machine 2.

## .4 Connection between the machines

Before starting the setup, it is essential to first establish a connection between the two machines which will host the 4G network components. This connection is crucial as it enables the exchange of control and user data between the EPC and the RAN. The machines are connected via a ethernet cable, and the necessary network configurations are applied to ensure proper communication (.3).

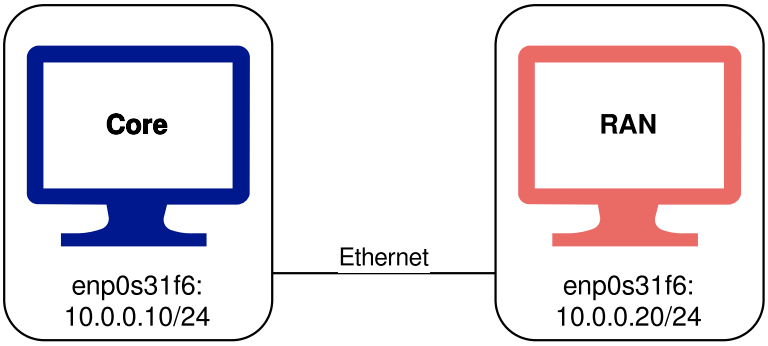


Figure .3: Diagram illustrating the connection between the two machines in the 4G network setup.

The steps to enable this connection are similar on both machines, differing only in the specific IP addresses assigned to each machine.

Connect both machines with an Ethernet cable and check the available network interfaces using the following command:

```
ip a
```

The output shows the available network interfaces, including the Ethernet interface (in this case enp0s31f6) (.4).

Figure .4: Output of the 'ip a' command, showing the available network interfaces, including the Ethernet interface (enp0s31f6).

Enable the Ethernet interface (if disabled) and assign a static IP address to it.

```
sudo ip link set enp0s31f6 up
sudo ip addr add 10.0.0.10/24 dev enp0s31f6
```

Core: 10.0.0.10/24 RAN: 10.0.0.20/24

To make the connection persistent across reboots, the network configuration needs to be added to the Netplan configuration file. This file may have a different name from the one in this example. Open the Netplan configuration file on `/etc/netplan/01-netcfg.yaml` using a text editor and add the following configuration to assign a static IP address to the Ethernet interface:

```
# Let NetworkManager manage all devices on this system
network:
  version: 2
  renderer: NetworkManager
  ethernets:
    enp0s31f6:
      dhcp4: false
      addresses:
        - 10.0.0.10/24
```

After saving the changes, apply the new network configuration using the following command:

```
sudo netplan apply
```

Finally, verify that the machines can communicate with each other by pinging the assigned IP addresses. From Machine 1:

```
ping 10.0.0.20
```

The output should show successful ping responses, indicating that the machines are connected and can communicate with each other (.5).

Figure .5: Output of the ping command, showing successful responses between the two machines, indicating that they are connected and can communicate with each other.

Having established a successful connection between the two machines, the next step is to proceed with the installation and configuration of the software components necessary for the 4G network setup.

## .5 Software Setup

### .5.1 Open5GS

This section provides a step-by-step guide for installing and configuring Open5GS. The installation process comprises of the following steps:

1. Installing MongoDB.
2. Installing Open5GS.
3. Installing the WebUI for Open5GS.
4. Configuring Open5GS components (MME, HSS, SGW, PGW) according to the network requirements.

The necessary steps and more information about the project and installation steps can be found in the official Open5GS documentation [2]. Unless otherwise specified, all steps should be executed on the machine designated to run the EPC (Machine 1 in our setup).

#### .5.1.1 Installing MongoDB

Open5GS depends on MongoDB as its database backend, where it stores subscriber information, network configurations, and operational data. Therefore, the first step in setting up Open5GS is to install MongoDB:

```
sudo apt update
sudo apt install gnupg curl -y
curl -fsSL https://pgp.mongodb.com/server-8.0.asc | sudo gpg -o
    /usr/share/keyrings/mongodb-server-8.0.gpg --dearmor

echo "deb [ arch=amd64,arm64
    signed-by=/usr/share/keyrings/mongodb-server-8.0.gpg ]
    https://repo.mongodb.com/apt/ubuntu jammy/mongodb-org/8.0 multiverse" |
    sudo tee /etc/apt/sources.list.d/mongodb-org-8.0.list

sudo apt update
sudo apt install -y mongodb-org
```

After executing the above commands, MongoDB will be installed on the machine. We can then start and verify the installation of the MongoDB daemon:

```
sudo systemctl start mongod
sudo systemctl status mongod
```

In which the output should indicate that the MongoDB service is active and running:

```
core@core:~$ sudo systemctl status mongod
● mongod.service - MongoDB Database Server
     Loaded: loaded (/lib/systemd/system/mongod.service; enabled; vendor preset: enabled)
     Active: active (running) since Tue 2026-04-21 17:41:12 WEST; 1 week 3 days ago
       Docs: https://docs.mongodb.org/manual
```

Figure .6: Output of the command to check MongoDB service status, showing that the service is active and running.

To ensure MongoDB starts automatically on system boot, we can enable the service with the following command:

```
sudo systemctl enable mongod
```

For more detailed instructions and troubleshooting guidance, refer to the official MongoDB installation documentation [1].

### .5.1.2 Installing Open5GS

After installing MongoDB, the next step is to add the Open5GS repository to the system and install Open5GS. This can be done by executing the following commands:

```
sudo add-apt-repository ppa:open5gs/latest
sudo apt update
sudo apt install open5gs -y
```

This will install the Open5GS package. After the installation is complete, we can verify that the Open5GS services are running correctly by checking their status:

```
sudo systemctl status open5gs-*
```

#### .5.1.3 Installing WebUI for Open5GS

Open5GS provides a WebUI for easier management of subscriber information, network configurations, and monitoring. The interface depends on Node.js, which needs to be installed first (on most Ubuntu builds, Node.js comes already pre-installed). To install Node.js, we can use the following commands:

```
sudo apt update
sudo apt install -y ca-certificates curl gnupg
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://deb.nodesource.com/gpgkey/nodesource-repo.gpg.key |
sudo gpg --dearmor -o /etc/apt/keyrings/nodesource.gpg

NODE_MAJOR=20
echo "deb [signed-by=/etc/apt/keyrings/nodesource.gpg]
    https://deb.nodesource.com/node_$NODE_MAJOR.x nodistro main" |
sudo tee /etc/apt/sources.list.d/nodesource.list

sudo apt update
sudo apt install nodejs -y
```

After installing Node.js, we can proceed to install the Open5GS WebUI by running the following command:

```
curl -fsSL https://open5gs.org/open5gs/assets/webui/install |
sudo -E bash -
```

The WebUI can be accessed by navigating to 'http://localhost:9999' in a web browser. The default login credentials are:

- Username: admin
- Password: 1423

### .5.1.4 Configuring Open5GS Components

After installing Open5GS and the WebUI, the next step is to configure the core network components (MME, HSS, SGW, PGW) according to the network requirements and adhering to the legal regulations. In Open5GS, the configuration files for these components are located in the `/etc/open5gs/` directory, with each component having its own file.

First modify the MME configuration file, on `/etc/open5gs/mme.yaml`, to set the PLMN ID, TAC and IP address to attach the MME. Unless issued one by the national state regulator, use the international test PLMN ID (001/01), which is the one used in this configuration.

```
--- a/configs/open5gs/mme.yaml.in
+++ b/configs/open5gs/mme.yaml.in
@@ -10,7 +10,7 @@ mme:
     freeDiameter: @sysconfdir@/freeDiameter/mme.conf
     s1ap:
       server:
-        - address: 127.0.0.2
+        - address: 10.0.0.10
     gtpc:
       server:
         - address: 127.0.0.2
@@ -25,14 +25,14 @@ mme:
           port: 9090
     gummei:
       plmn_id:
-        mcc: 999
-        mnc: 70
+        mcc: 001
+        mnc: 01
       mme_gid: 2
       mme_code: 1
     tai:
       plmn_id:
-        mcc: 999
-        mnc: 70
+        mcc: 001
+        mnc: 01
       tac: 1
     security:
         integrity_order : [ EIA2, EIA1, EIA0 ]
```

Modify the SGWU configuration file, on `/etc/open5gs/sgwu.yaml`, to set the GTP-U

IP address.

```
--- a/configs/open5gs/sgwu.yaml.in
+++ b/configs/open5gs/sgwu.yaml.in
@@ -15,7 +15,7 @@ sgwu:
 #          - address: 127.0.0.3
     gtpu:
       server:
-        - address: 127.0.0.6
+        - address: 10.0.0.10
```

After modifying the configuration files, restart the Open5GS daemons to apply all changes:

```
sudo systemctl restart open5gs-mmed
sudo systemctl restart open5gs-sgwud
```

Having completed the configuration of the Core components, the next step is to proceed with the installation and configuration of the RAN components, which will be covered in the next section.

### .5.2 SDR Configuration

Before proceeding with the installation of srsRAN 4G, it is necessary to first configure the Nuand BladeRF xA4 SDR and install the necessary drivers to bridge the connection between the software and the hardware. This procedure consists of the following steps:

1. Setting up the Nuand drivers and firmware for the BladeRF xA4, with header files.
2. Installing SoapySDR.

#### .5.2.1 Setting up the Nuand drivers and firmware for the BladeRF xA4

Nuand provides official Ubuntu package repositories (PPAs) that simplify the installation of the BladeRF software in Ubuntu systems. Through these repositories, users can quickly install both the required host tools and libraries and the necessary firmware for the FPGA using `apt`, without needing to build everything manually from source. To enable the PPA:

```
sudo add-apt-repository ppa:nuandllc/bladerf
sudo apt update
```

After enabling the PPA, we can install the BladeRF software and firmware using the following command:

```
sudo apt install bladerf
sudo apt install bladerf-fpga-hostedx40
```

To bridge the connection between the BladeRF xA4 and the RAN, SoapySDR will be used, which will be covered in the next section (.5.2.2). For this to work, the header files for the BladeRF drivers need to be installed:

```
sudo apt install libbladerf-dev
```

With the BladeRF drivers installed, we can proceed into the installation of SoapySDR, which will allow us to use the BladeRF xA4 as the radio interface for the RAN.

### .5.2.2 Installing SoapySDR

The SoapySDR component is comprised of two parts: the SoapySDR library, which is the main interface for all types of SDR devices, and the SoapyBladeRF plug-in, which provides specific support for the BladeRF xA4.

First, we need to install the necessary dependencies:

```
sudo apt install -y cmake g++ libpython3-dev python3-numpy swig
```

Afterwards, we can clone the SoapySDR repository and build the library:

```
git clone https://github.com/pothosware/SoapySDR.git
cd SoapySDR

mkdir build
cd build
cmake ..
make -j`nproc`
sudo make install -j`nproc`
sudo ldconfig
```

To verify the installation of the SoapySDR library, we can run the following command:

```
SoapySDRUtil --info
```

After installing SoapySDR, we can proceed to install the SoapyBladeRF plug-in, through their repository:

```
git clone https://github.com/pothosware/SoapyBladeRF.git
cd SoapyBladeRF

mkdir build
cd build
cmake ..
make
sudo make install
```

With both the SoapySDR library and the SoapyBladeRF plug-in installed, the BladeRF xA4 should now be recognized as an available SDR device, when running the following command:

```
SoapySDRUtil --probe="driver=bladerf"
```

This command should output the details of the board, thus confirming that the BladeRF xA4 is properly configured and ready to be used as the radio interface for the RAN in our 4G network setup. With the SDR configuration complete, we can now proceed to install and configure srsRAN 4G.

### .5.3 srsRAN 4G

First, install the dependencies for srsRAN 4G:

```
sudo apt-get install build-essential cmake libfftw3-dev libmbedtls-dev
    libboost-program-options-dev libconfig++-dev libsctp-dev
```

Then, clone the srsRAN repository and build the software:

```
git clone https://github.com/srsRAN/srsRAN_4G.git
cd srsRAN_4G
mkdir build
cd build
cmake ../
make
make test
```

Proceed to install srsRAN 4G:

```
sudo make install
srsran_install_configs.sh user
```

After installing srsRAN 4G, we need to modify the configuration files to set the correct IP address for the EPC and the correct parameters for the radio interface. The configuration files are located in the `/usr/local/etc/srsran/` directory, with separate files for the EPC and the eNodeB. With everything ready, we can proceed to run the network, which will be covered in the next section (.6).

## .6 Running the network

### .6.1 Preparing the system

To ensure the best performance on the RAN side, the srsRAN team provices a performance script to optimize the system for real-time processing. This script should be executed before running the RAN components, on Machine 2:

```
# Download and execute the srsRAN performance tuning script
curl -fsSL
    https://raw.githubusercontent.com/srsran/srsRAN_Project/main/scripts/srsran_performance
    | bash
```

On Machine 1, we need to add a route to the UE subnet, so that the EPC can route the traffic from the UE to the internet:

```
### Enable IPv4/IPv6 Forwarding
$ sudo sysctl -w net.ipv4.ip_forward=1
$ sudo sysctl -w net.ipv6.conf.all.forwarding=1

### Add NAT Rule
$ sudo iptables -t nat -A POSTROUTING -s 10.45.0.0/16 ! -o ogstun -j
    MASQUERADE
$ sudo ip6tables -t nat -A POSTROUTING -s 2001:db8:cafe::/48 ! -o ogstun -j
    MASQUERADE
```

Now, to start the network, we need to first start the EPC on Machine 2. On one command prompt window, run the following command:

```
sudo srsepc
```

Machine 2 should now connect to Machine 1, and srsRAN should recognize the Open5GS network functions running on the other Machine. In another command prompt window, start the eNodeB on Machine 2:

```
sudo srsenb
```

Now the RAN is up and running, and the UE can connect to the network.

## .7 Troubleshooting

# Provisioning the eSIM and connecting to the network

In this chapter, we will provision an eSIM profile for the testbench and connect to the network. We will create an eSIM profile through a provisioning server and follow the steps to activate it on our testbench. Once the eSIM is activated, we will verify the connection to the network and ensure that we can send and receive data. This process is crucial for testing the performance of our system under real network conditions.

## .8 Creating an eSIM profile

To create an eSIM profile for our testbench, we will use Simlessly, specifically their Remote SIM Provisioning (RSP) tool, which allows custom eSIM profile creation. This platform requires an account to access the provisioning services. Once logged in, the user is greeted with a dashboard:

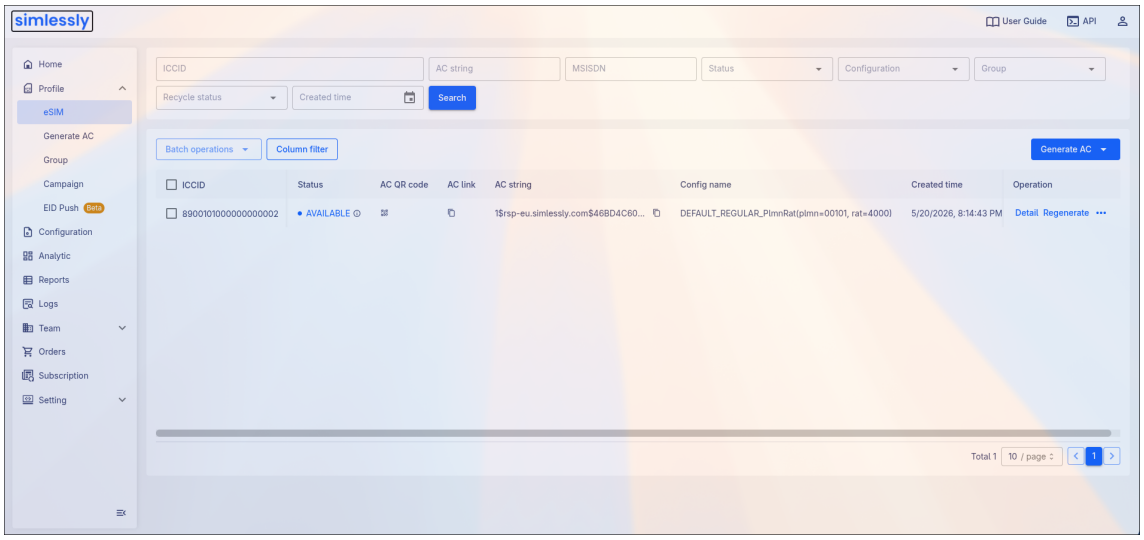


Figure .7: Simlessly Dashboard

From the dashboard, we can click the "Generate AC" button to create a new eSIM profile.

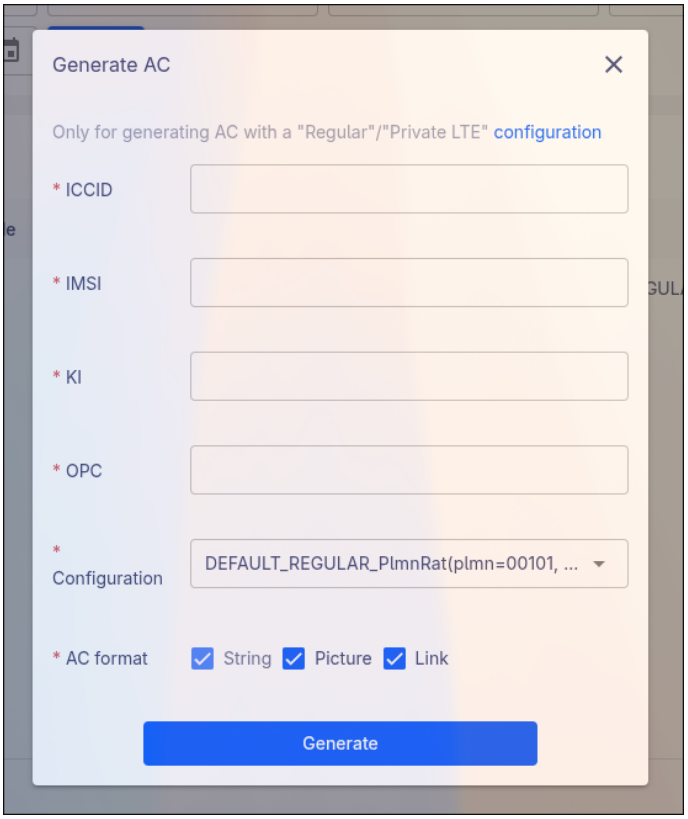


Figure .8: Generating an eSIM profile

After clicking the button, we are prompted to fill in the details for the eSIM profile. For the testbench, we will use the following keys:

| **Key** | **Value** |
|---|---|
| ICCID | 8900101000000000002 |
| IMSI | 101000000000002 |
| KI | 465B5CE8B199B49FAA5F0A2EE238A6BC |
| OPC | E8ED289DEBA952E4B4C389C94229606A |

Table .4: eSIM profile details

Click on "Generate" to create the eSIM profile. Once generated, back on the dashboard, click on the QR code icon on the "AC QR code" column to display the Activation Code:

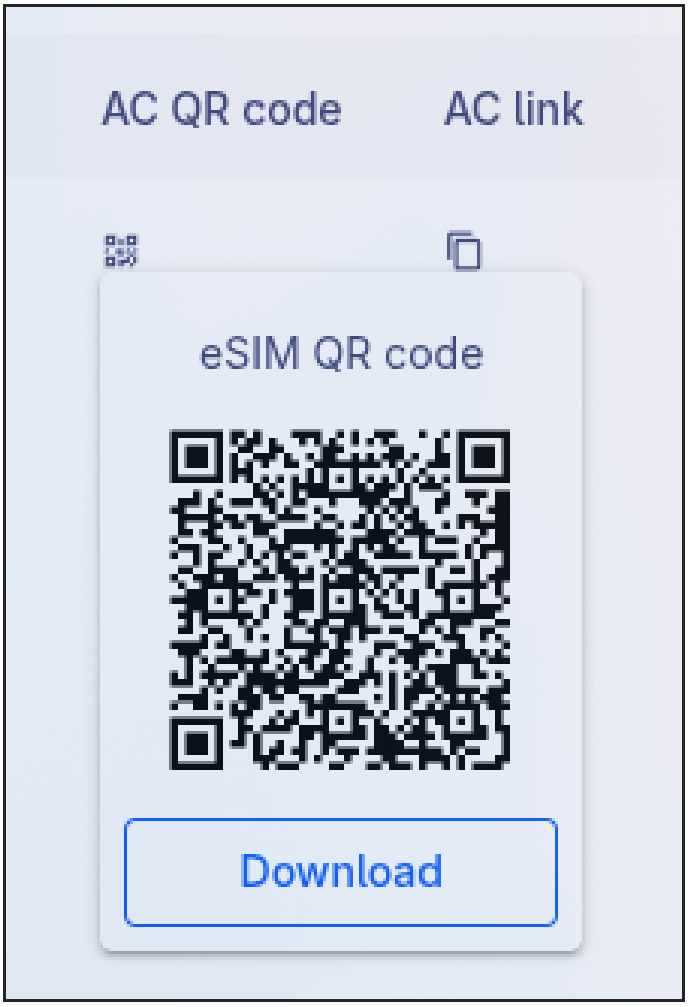


Figure .9: eSIM Activation Code